Caso de Estudio Cruz Roja

Anyela Lineth Cabrera Ordoñez
202540031
anyela.cabrera@correounivalle.edu.co

Carlos Anibal Ceron del Castillo
202537097
carlos.ilich.ceron@correounivalle.edu.co

Javier Felipe Contreras Díaz
202535936
contreras.javier@correounivalle.edu.co

Facultad de Ingeniería, Universidad del Valle

750015C: Fundamentos de Programación Orientada a Objetos

Victor Andres Bucheli Guerrero

18 de marzo de 2025

Preguntas De Análisis, Caso De Estudio Cruz Roja

Evaluación de las ventajas de la codificación en el Paradigma Orientado a Objetos:

1. Modularidad: ¿Cuáles módulos o clases identifica en el caso? ¿Considera que cada clase representa un objeto más pequeño y manejable? ¿En cuál sección del código encuentra que el programa está dividido?

Para este caso, vemos como el proyecto se ve dividido en dos clases principales y un main; una clase Donor que define la creación de objetos (newDonor) para cada donante registrado. Dentro del objeto donante guarda algunos valores sociodemográficos (Teléfono, dirección, departamento)y el tipo de sangre del donante. Mientras que la clase BloodDatabase hace gestión de los métodos, registro, búsqueda, lectura o eliminación de los datos referente a los atributos de Donor. A la vez que administra toda la colección de objetos tipo Donor creados. El main sirve de interfaz y brinda el flujo de la ejecución. No obstante debido a cómo está estructurado el proyecto, el main no se aprovecha al máximo para gestionar el flujo del programa, por lo que su principal utilidad es la interacción directa con el usuario a través del menú principal y da estructura al proyecto para la compilación.

2. Evalúe si la siguiente expresión es verdadera o falsa y justifique su respuesta: La modularidad facilita la localización y corrección de errores. Si hay un problema en una clase, se puede abordar sin afectar al resto del sistema.

Correcto. La modularidad facilita localizar y corregir errores, dada la forma en cómo se organiza el proyecto es posible generar una estructura modular y graficarla en un diagrama para su interpretación.
Al construir el código basado en la estructura modular y teniendo un mapa (diagrama) de las funciones y responsabilidades de cada clase y método es posible aislar los problemas y brindar soluciones a errores concretos. Según Acera (2017) la programación modular se basa en la idea de “Divide y vencerás”. La estrategia es dividir el programa en otros más sencillos llamados módulos, cada módulo tiene tareas sencillas y en caso de un fallo será más fácil modificar únicamente el módulo, que un programa más extenso.
Pese a esto, y que el actual programa tiene la división de tareas, sería posible mejorar la estructura, aislando más métodos para acortar los actuales, y mejorando el flujo de datos del main con las diferentes clases.

3. Evalúe si la siguiente expresión es verdadera o falsa y justifique su respuesta: Los desarrolladores pueden trabajar en diferentes clases simultáneamente sin conflictos, mejorando la productividad y la eficiencia del equipo de desarrollo

Siguiendo con la idea previa, el objeto de generar una estructura modular y además graficarla en un diagrama, permite tener los planos de un programa con partes, responsabilidades y funciones específicas. Esto permite la división del trabajo, en el

que cada persona puede aportar con una parte del programa (un módulo) y al final unir las diferentes piezas logrando completar el programa Acera (2017).

4. Reusabilidad: ¿Qué código ha reutilizado hasta el momento?

En este caso, para la ejecución de la guía reutilicé el código de display.Distrtic() para crear el display.bloodType() generando una cadena de opciones y modificando según las necesidades especificas del tipo de sangre. El código utiliza varias veces la verificación y el procesado de los datos con el método parseLine. En BloodType podemos ver como se reutiliza el método de validación de datos numericos.

5. Evalúe si la siguiente expresión es verdadera o falsa y justifica tu respuesta: Las clases pueden ser reutilizadas en diferentes partes del programa o incluso en otros proyectos, reduciendo el esfuerzo de desarrollo.

Verdadero. En el proyecto actual, se observa que la clase Donor funciona como un componente independiente que encapsula los datos del donante. Esta estructura permite que la clase sea "portátil"; por ejemplo, los archivos Donor.h y Donor.cpp podrían integrarse en un sistema de gestión hospitalaria diferente sin necesidad de modificar su lógica interna. Al separar la entidad (Donante) de la gestión (Base de Datos), el programador no tiene que reescribir el código de modelado de datos en cada nuevo proyecto, cumpliendo así con uno de los pilares fundamentales de la POO que es el ahorro de tiempo y recursos.

6. Evalúe si la siguiente expresión es verdadera o falsa y justifique su respuesta:
El ocultamiento de la información permitió resolver los requerimientos del caso.

El ocultamiento, aplicado mediante el uso de atributos private en las clases, es lo que garantiza la integridad de la base de datos de donantes. Al evitar que el main o funciones externas modifiquen directamente variables críticas como donorId o bloodType, el programa obliga a pasar por métodos controlados como getDonorDetails() o los setters. Esto resolvió el requerimiento de mantener una base de datos confiable, ya que asegura que la información solo se altere bajo las reglas definidas por el desarrollador, evitando errores humanos de manipulación directa durante la ejecución.

7. ¿Considera que en el caso hay consistencia, pues la reutilización de código asegura que las funcionalidades compartidas se comporten de manera uniforme en todo el sistema?

Sí, existe una consistencia clara. Se evidencia una uniformidad operativa gracias al uso de métodos compartidos. Un ejemplo clave es el método Donor::parseLine(), el cual es utilizado tanto en la búsqueda de donantes, como en la generación del historial y en el proceso de eliminación. Al reutilizar esta misma lógica de lectura en

diferentes secciones, el sistema garantiza que el archivo data.txt siempre se interprete de la misma forma. Si no existiera esta reutilización, el programa correría el riesgo de leer un dato en un formato para la búsqueda y en otro distinto para el historial, generando fallos de concordancia.

8. Extensibilidad: ¿Qué nuevas funcionalidades adiciona a la aplicación? ¿considera que es fácil añadir nuevas funcionalidades y extender las existentes?

Para mejorar el alcance actual, se podrían adicionar funcionalidades como:

- Búsqueda por ID único: Para evitar confusiones entre donantes con nombres similares.
- Validación de fechas de donación: Un campo que indique cuándo fue la última vez que donó la persona para calcular si es apta nuevamente.
- Generador de reportes: Una función que exporte la lista de donantes a un formato más legible como CSV o PDF.

Considero que el sistema es fácil de extender debido a su arquitectura basada en clases. Gracias a que las responsabilidades están separadas, si quisiéramos añadir un nuevo atributo (como "Edad" o "Peso"), solo tendríamos que modificar la clase Donor y los métodos de persistencia en BloodDatabase. El flujo principal del main no se vería comprometido, lo que demuestra que la estructura modular permite que el programa crezca de forma orgánica sin necesidad de reconstruirlo desde cero.

9. ¿Considera que el código del caso es claro y organizado? ¿La Mantenibilidad facilita la comprensión y modificación del sistema?

El código presenta una organización básica mediante la creación de 2 clases definidas; sin embargo, estas no cumplen completamente con el principio de responsabilidad única, ya que se agregan funcionalidades que se verían mejor modeladas en una clase para utilidades. Esto impacta directamente en la mantenibilidad y modificación del sistema, donde la sobrecarga de funciones hará mucho más difícil el proceso de debugging. Adicionalmente, hay limitados comentarios detallando la función de cada método y su relación con las demás clases, sin estos, es necesario revisar línea por línea el significado sintáctico del código.

10. ¿Considera que el código del caso es sencillo de actualizar y mejorar? ¿Esto se logra por la separación de responsabilidades entre las clases?

Aunque el sistema se puede actualizar, la separación entre las funciones de cada clase no es la correcta y a medida que se implementen nuevas funciones y métodos a cada clase, se puede generar confusión. Ejemplo de esta situación se evidencia al interior de la clase Donor y la clase BloodDatabase.

En Donor se incluye el método trim, el cual manipula cadenas de texto. Para cumplir de forma correcta con el principio de responsabilidad única de la clase, este método

encajaría en una nueva clase de utilidades, lo cual permitirá ubicarlo de forma más intuitiva y corregir cualquier error derivado de su implementación.

BloodDatabase debe centrarse en la gestión de los datos de cada donante, pero se incluyen métodos para validación de inputs y para limpiar la consola. Este conjunto, sobrecarga a la clase de responsabilidades que no corresponden a un modelado correcto.

11. Facilidad de comprensión ¿El código entregado en el caso está escrito de la forma en que nosotros entendemos y modelamos el mundo real? ¿Es intuitivo el uso de objetos y relaciones entre ellos?

El código contiene algunos elementos modelados a partir del mundo real, como lo son los atributos de la clase donor, guardando información relevante respecto a registrar un donante. Sin embargo, métodos como trim() no hacen parte de un modelado correcto. Al realizar el análisis de modelado sobre bloodDatabase, encontramos que validar entradas y limpiar consola no corresponden a responsabilidades de la clase. A pesar de estos inconvenientes, el uso de estas dos clases permite que la interacción entre sus componentes sea intuitiva al momento de entender y mejorar el código.

12. Se puede argumentar que:

El software proporciona resultados correctos o precisos.

El sistema produce resultados correctos teniendo en cuenta el alcance actual del programa. Esto teniendo en consideración que el inconveniente con la variable number fue solucionado seleccionando el tipo de dato correcto. El código cumple con la siguiente funcionalidad:

1. Guarda información de los donantes en un archivo.
2. Busca en el archivo de guardado la información registrada por el usuario respecto de cada donante.
3. Es capaz de eliminar donantes modificando el archivo.

Adicionalmente, el programa es capaz de realizar validaciones del tipo de dato al momento de ser ingresado por el usuario, tanto al momento de guardar como de buscar, asegurandose que este sea un entero para las variables correspondientes e indicando un error en caso contrario; sin embargo, no existe una forma de verificar que el rango del dato ingresado cumpla con las opciones dadas (el usuario puede ingresar un número mayor al de las opciones brindadas para departamento y tipo de sangre).

Cabe la pena mencionar que en un entorno profesional no sería adecuado guardar los datos en un archivo de texto plano, el cual puede ser sujeto de manipulación, errores de guardado o corrupción.